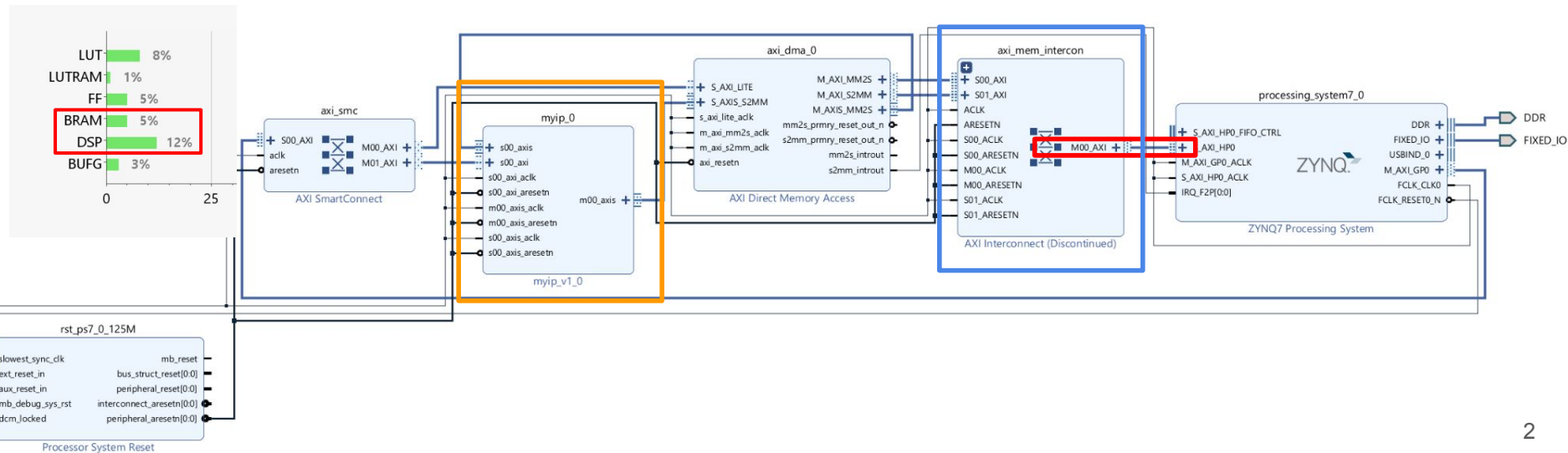


AXI 기반 3x3 convolution IP

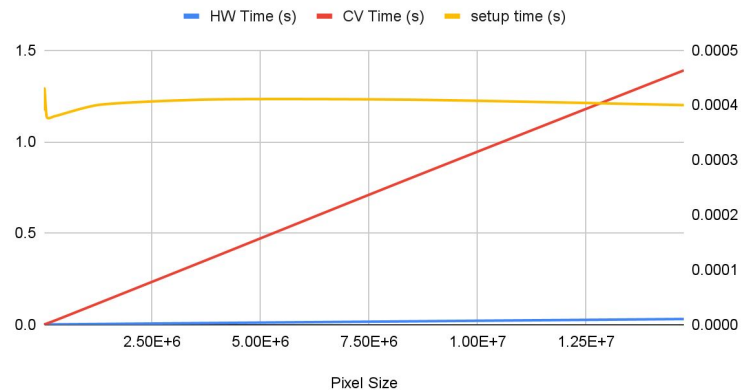
전자정보공학부
20211513 김동준

- AXI based 3x3 RGBA image convolution IP
- 4-stage pipeline architecture
- AXI-Lite(control register), AXI-Stream(Data flow), AXI DMA

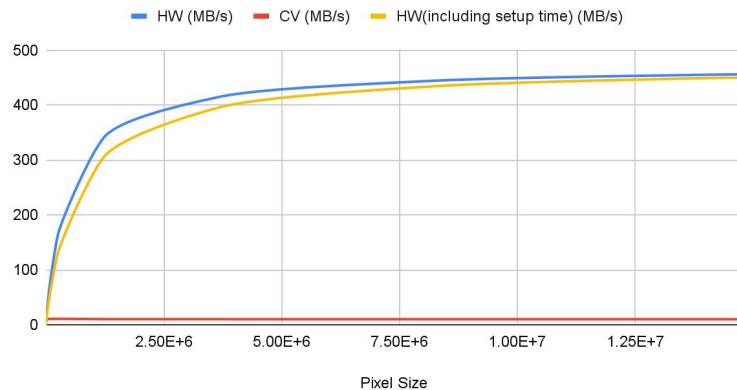


- Implemented on PYNQ-Z2 (zynq 7020), Pynq environment
- Up to x44 performance than OpenCV.filter2D()

HW vs CV time

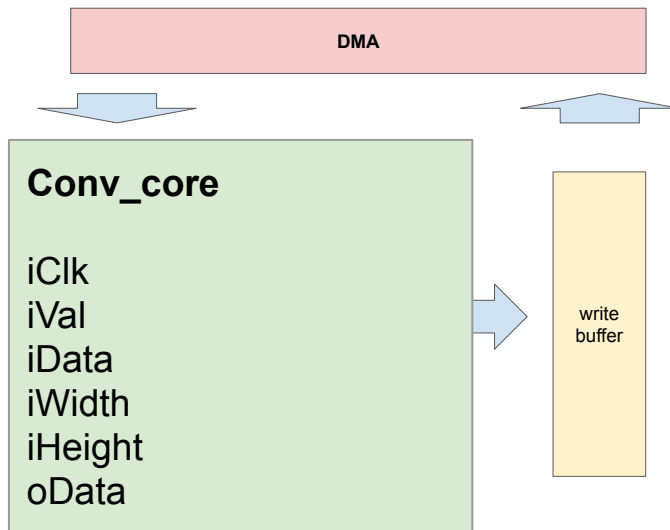


Throughput



Hierarchy : Convolution core

- Data streaming (minimal control plane)
- Flexibility
- Low latency



Abstraction : buffer as FIFO

- SPSRAM + Pointer register

FIFO

iCLK

iCSN

iWrn

iRstPtr

oDATA

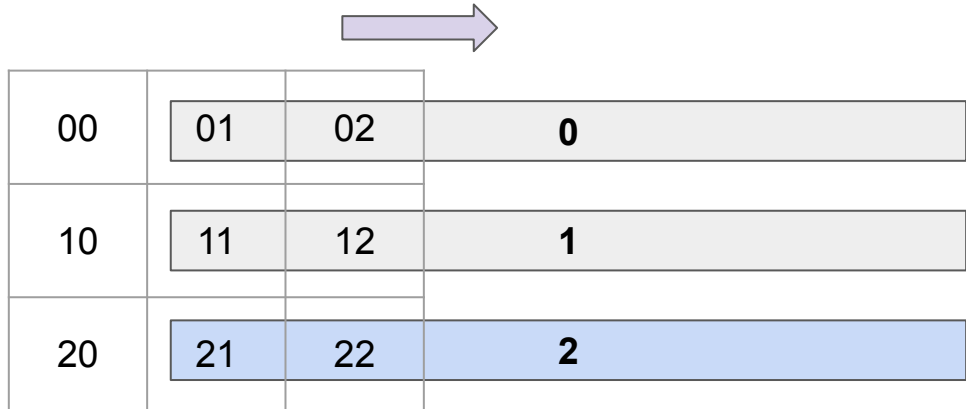
iDATA

2

High performance convolution module

Triple Line buffer for 3x3 convolution

- To maximize performance, Stream execution
- using Sliding window

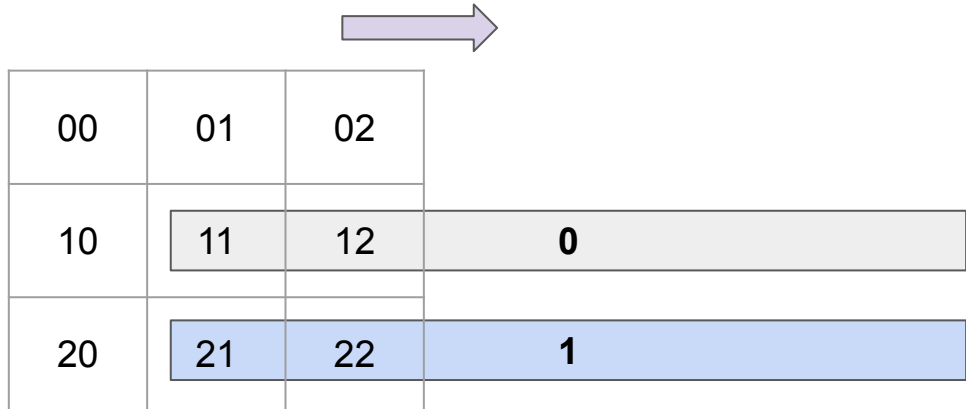


2

High performance convolution module

Triple Line buffer for 3x3 convolution

- **First Execution** at 2nd Line reading (including zero padding)



2

High performance convolution module

Triple Line buffer for 3x3 convolution

- **n - 1 th line Execution** ends at last data read

	00	01	02
n - 1	10	11	12
n	20	21	22

2

High performance convolution module

Triple Line buffer for 3x3 convolution

- Need extra cycle for last execution

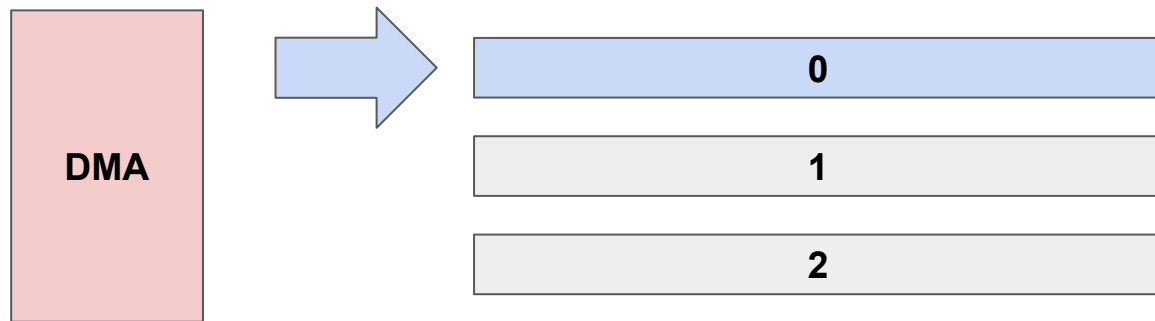
n - 1	00	01	02
	10	11	12
n			
	20	21	22

2

High performance convolution module

Triple Line buffer for 3x3 convolution

- 1 Write, 2 Read
- Buffer Round robin

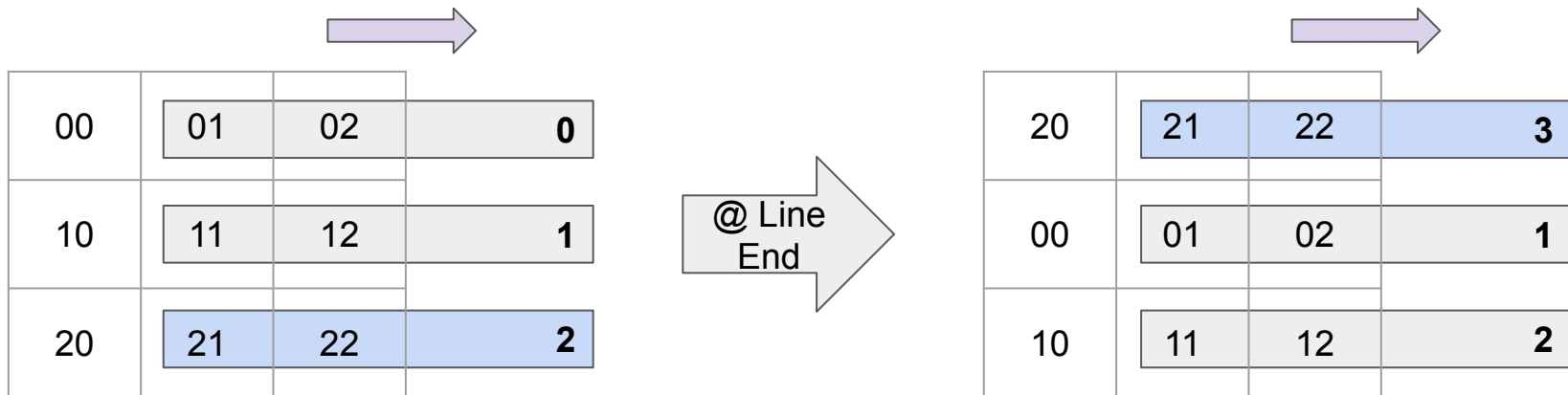


2

High performance convolution module

Triple Line buffer for 3x3 convolution

- Coeff update approach
($ak + bl + cm = bl + cm + ak$, property of summation)
- fixed calculation path

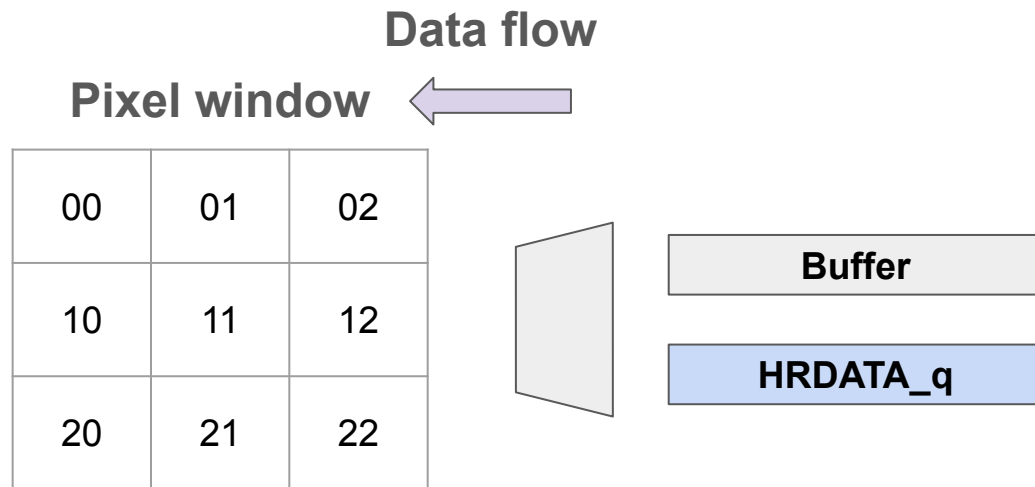


2

High performance convolution module

Path for 3x3 convolution

- 3x3 Shift register
- Stream Data
- Need mux



2

High performance convolution module

Path for 3x3 convolution

- MAC

Pixel window

00	01	02
10	11	12
20	21	22

*

Filter

00	01	02
10	11	12
20	21	22

=

Result

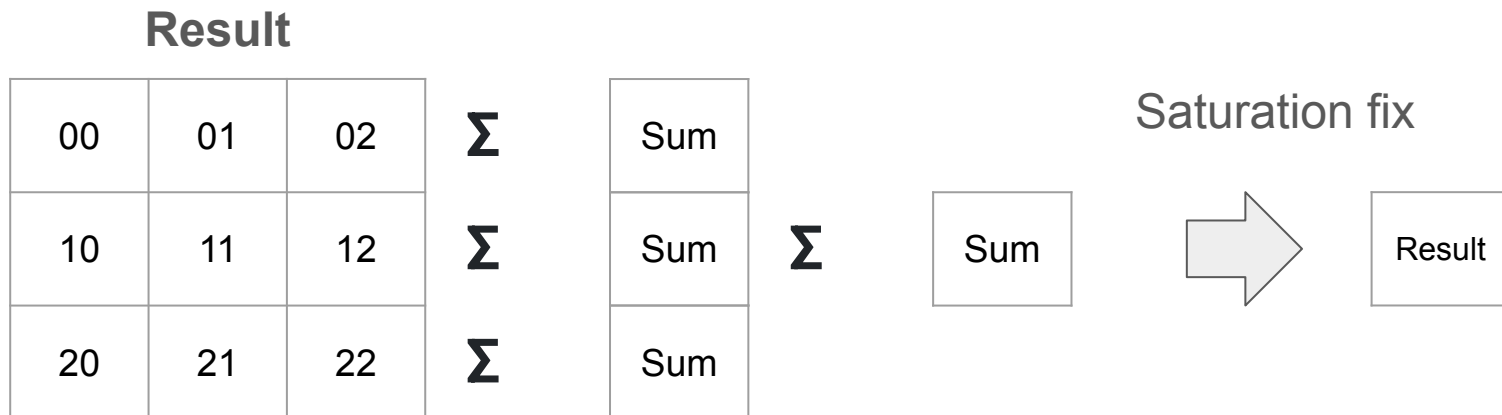
00	01	02
10	11	12
20	21	22

2

High performance convolution module

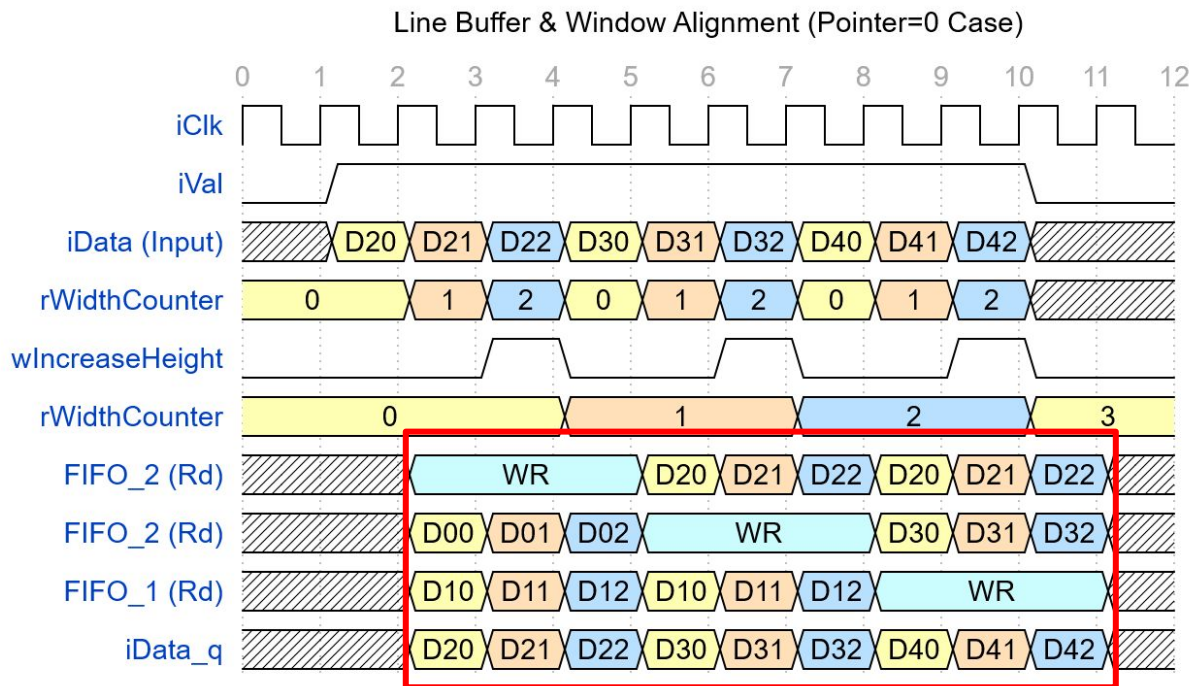
Path for 3x3 convolution

- Summation & saturation



Timing design

- one line latency
(buffer filling)
- one cycle latency



* Note: delayed by 1 clock due to SRAM Read Latency

Diagram : Conv_core

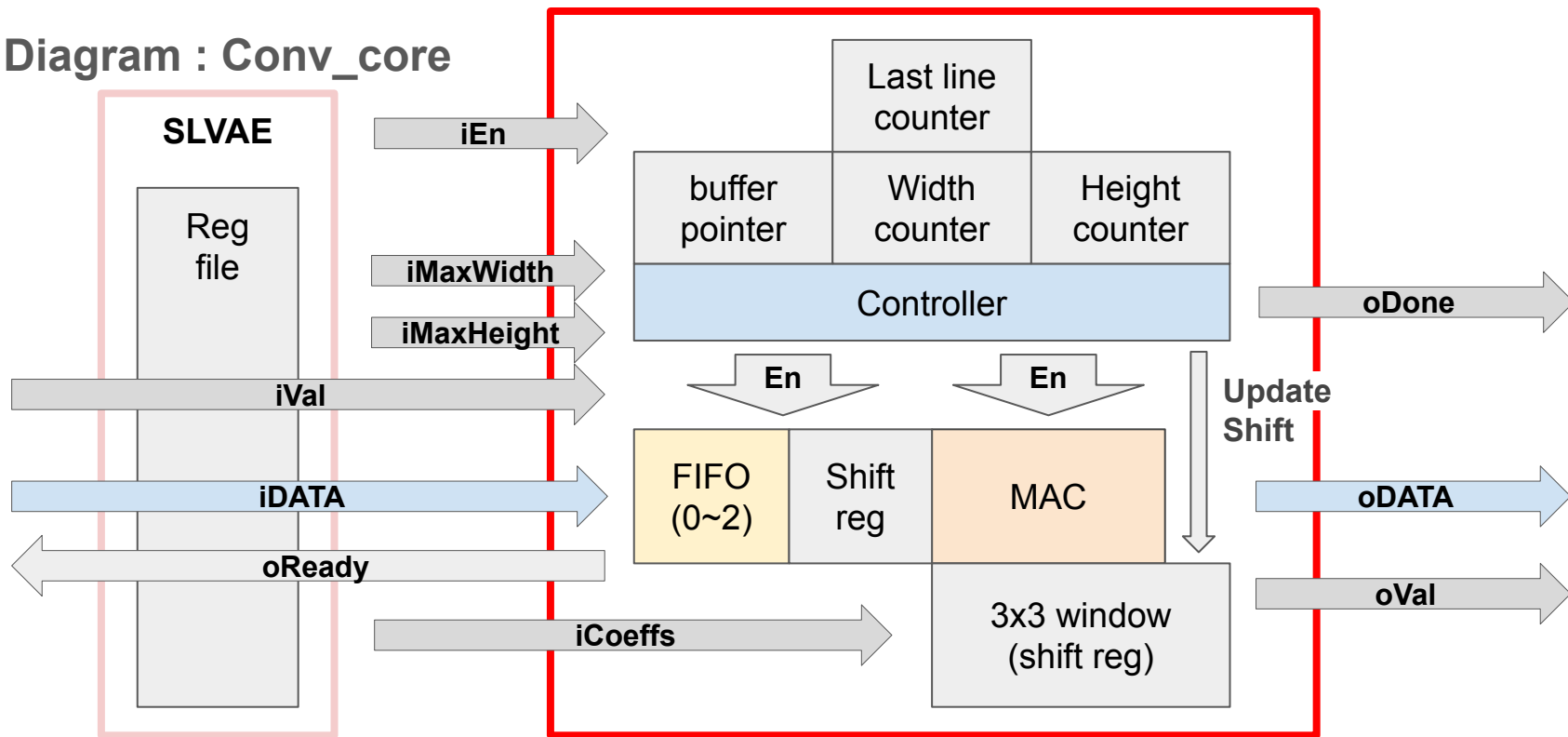
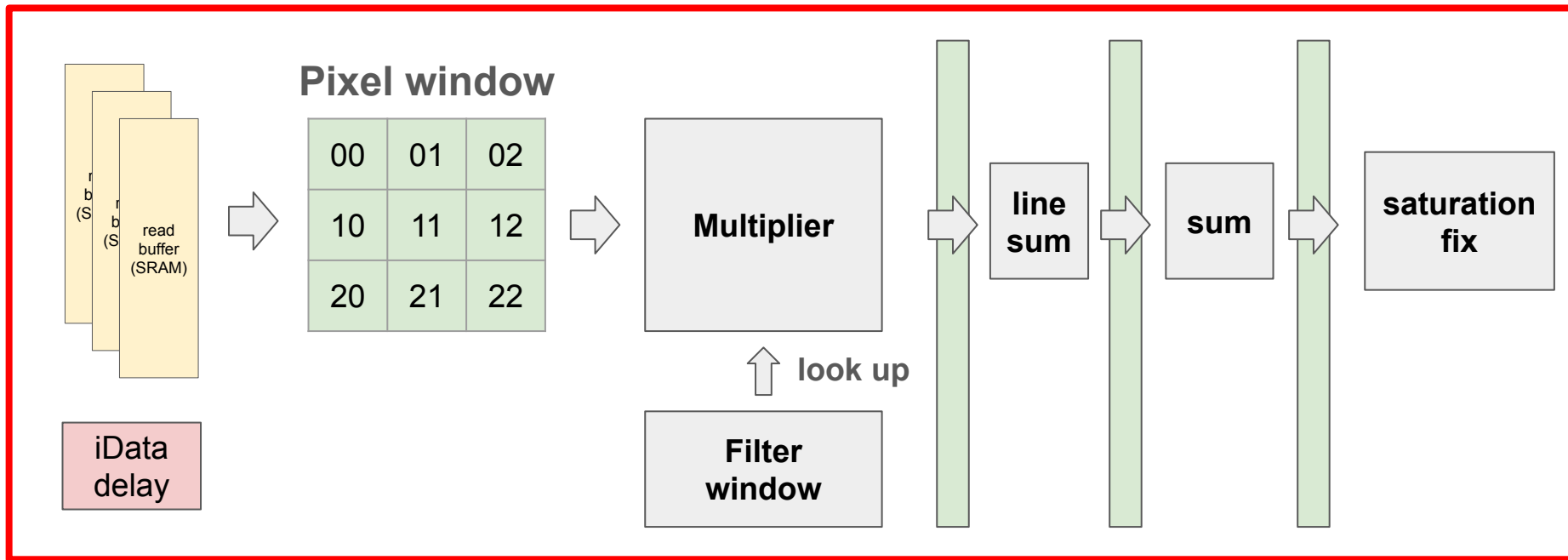


Diagram : Data pipeline



4 Build

Zero padding mux

- Edge case : end of line
- benefit: pre loading
=> No pipeline bubble for padding

0	01	02	30	3
1	11	12	10	1
2	21	22	20	2

Pixel window

01	02	30
11	12	10
21	22	20



padded window

00	01	0
10	11	0
20	21	0

*

Filter

00	01	02
10	11	12
20	21	22

4 Build

Zero padding mux

- Edge case : start of line
- benefit: mask old data
=> No pipeline bubble for flushing

0	02	30	31	3
1	12	10	11	1
2	22	20	21	2

Pixel window

02	30	31
12	10	11
22	20	21



padded window

0	30	31
0	10	11
0	20	21

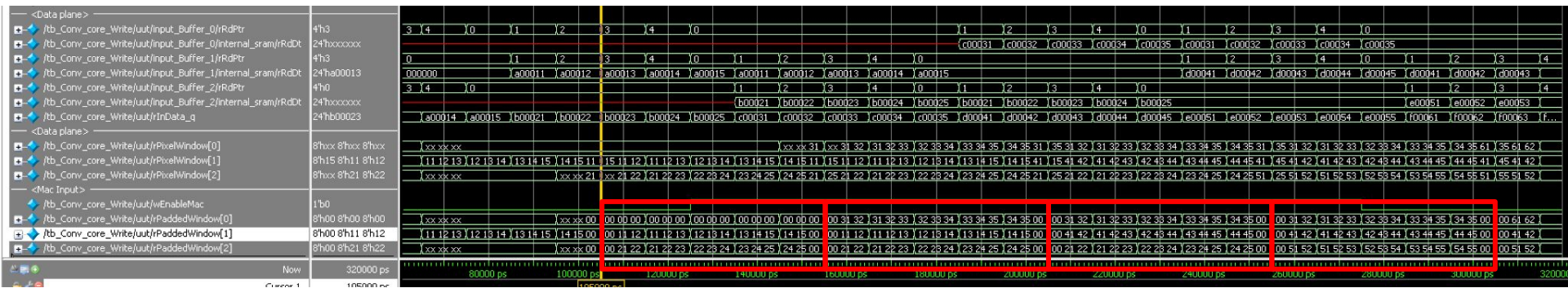
*

Filter

20	21	22
00	01	02
10	11	12

4 Build

Waveform



4 Build

4 stage pipeline Convolution + ReLu

- Add register slice for calculation path (maybe critical path)
- **Shift regl** - padding mux - Mul - **FF** - Adder tree - **FF** - Adder tree - **FF** - Saturation mux - Out

Pixel window

00	01	02
10	11	12
20	21	22

*

Filter

00	01	02
10	11	12
20	21	22

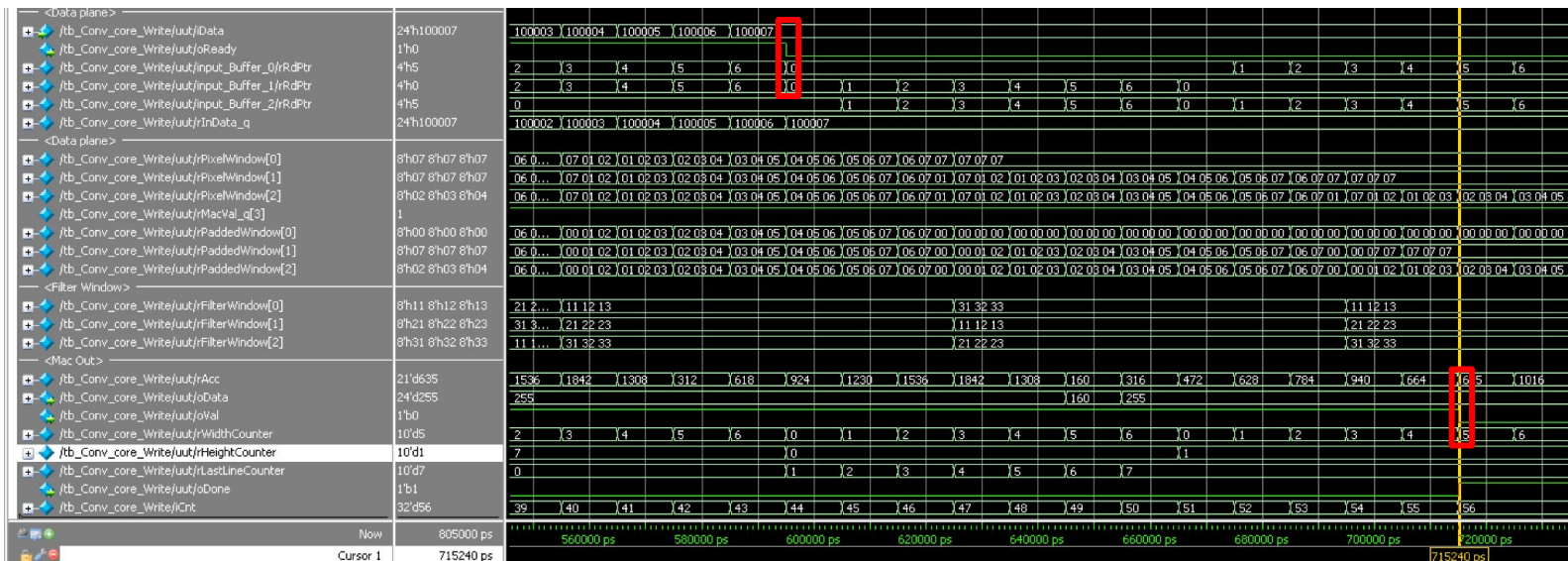
=

Result

00	01	02
10	11	12
20	21	22

5 Fix

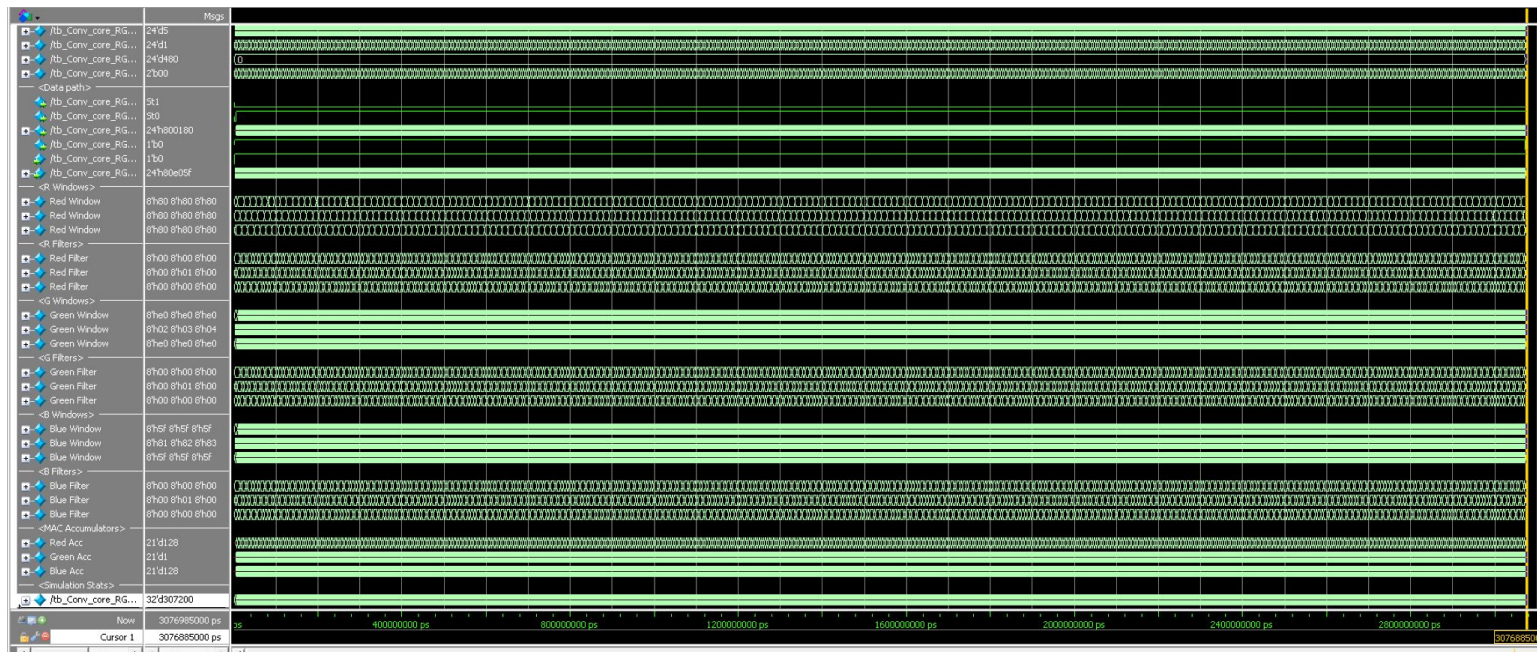
Waveform



5 Fix

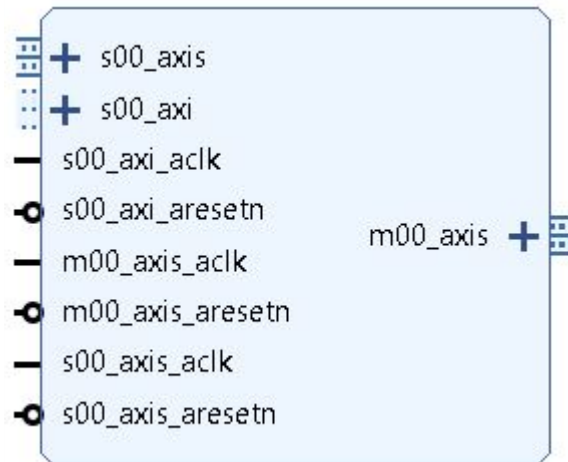
Testcase: RGBA, 3 channel Convolution, 480 x 640, Identity Filter

- data plane: No error detected



Bus Wrapper design

- Data stream: AXI stream
- Control register: AXI Lite
- Use external DMA



AXI stream protocol

- stream data sequentially without address
- handshake process - TVALID & TREADY
- via DMA

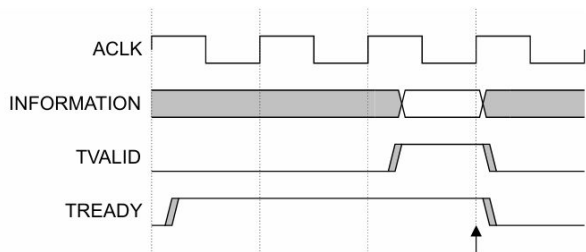


Figure 2-2 TREADY before TVALID handshake

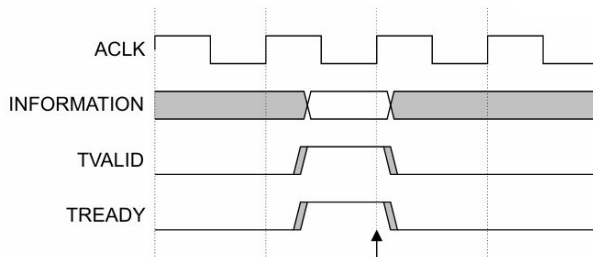


Figure 2-3 TVALID with TREADY handshake

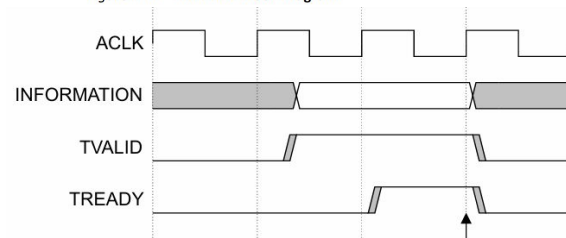


Figure 2-1 TVALID before TREADY handshake

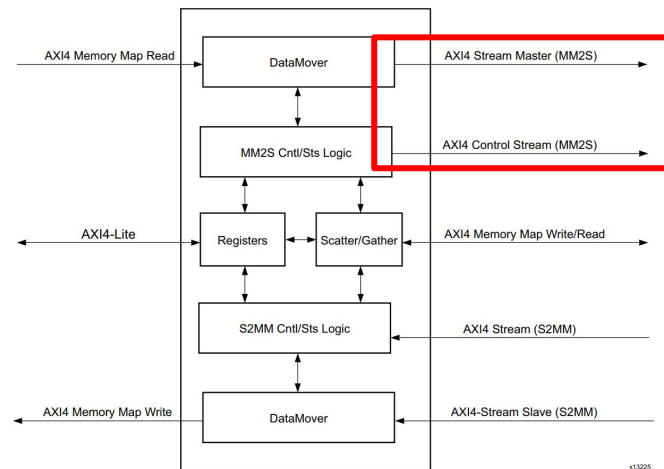


Figure 1-1: AXI DMA Block Diagram

x13025

Data plane pipeline halt

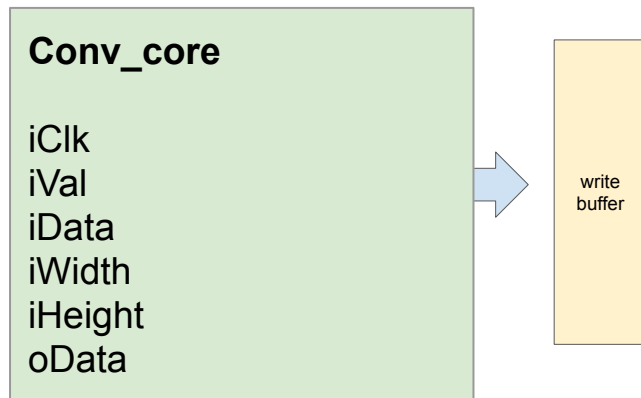
- Data driven pipeline on Conv_core

need to stop pipeline at

1. input invalid
 2. output not ready
- => *can cause deadlock

Output Queue

- Preventing deadlock in data-driven pipeline
deadlock when AXIS slave (DMA S2MM) has tReady after Valid handshake
- 32 x16 Register, circular queue
faster access, too small for SRAM



Fixed Pipeline halt condition

1. Output queue full
2. Invalid input

AXI-stream handshake signal

1. s00_axis_tready low when Output queue is full
2. m00_axis_tvalid low when Output queue is empty
3. m00_axis_tlast when end of data transfer

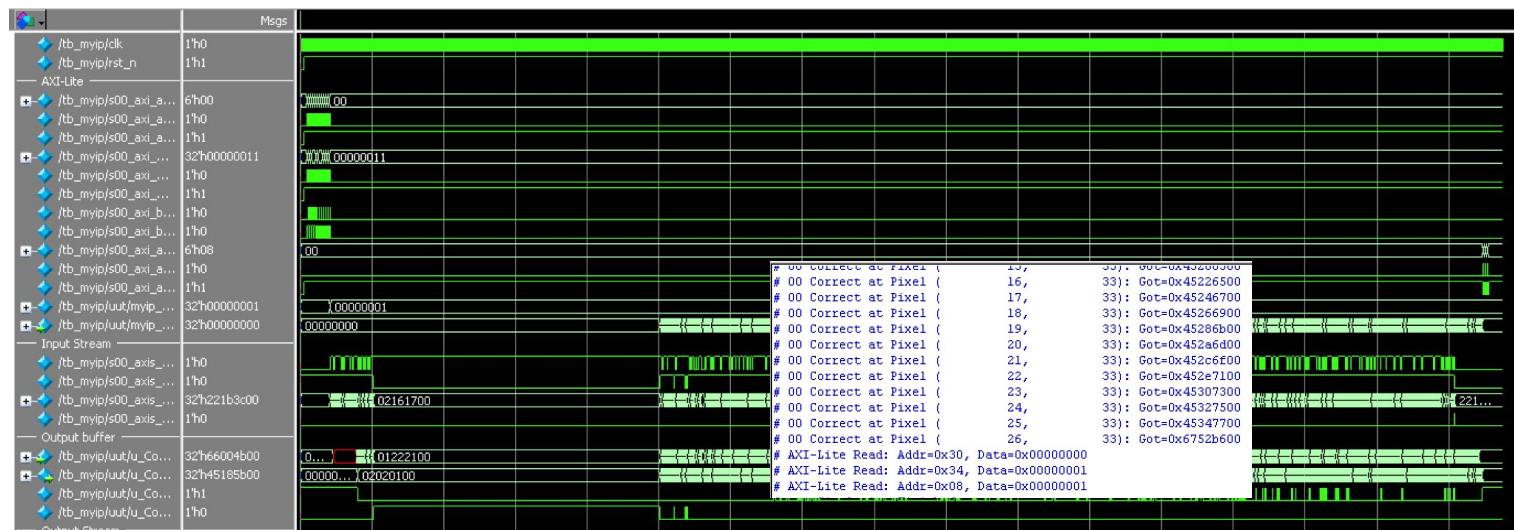
6 Wrapper design

Testbench : RGBA, random invalid input, output not ready, Identity filter

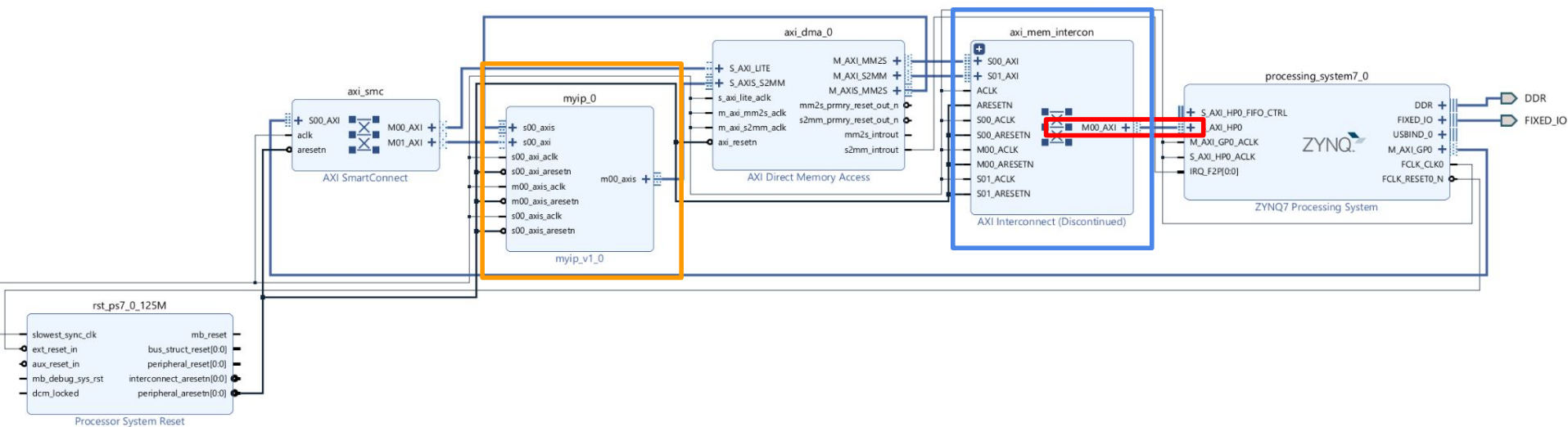
1. Halt when Queue full
2. No deadlock



Testbench : RGBA, random invalid input, output not ready, edge detect filter



Block design on vivado



6

Wrapper design

Registers Map - AXI-Lite Slave

offset	name	31 : 28	27 : 24	23 : 20	19 : 16	15 : 12	11 : 8	7 : 4	3 : 0
0x00	Config							[4] Reset (self flush)	[0] En
0x04	Size	[31:16] height				[15:0] width			
0x08	Status								[0] Done
0x0C ~	Coeff			Coeff02		Coeff01		Coeff00	

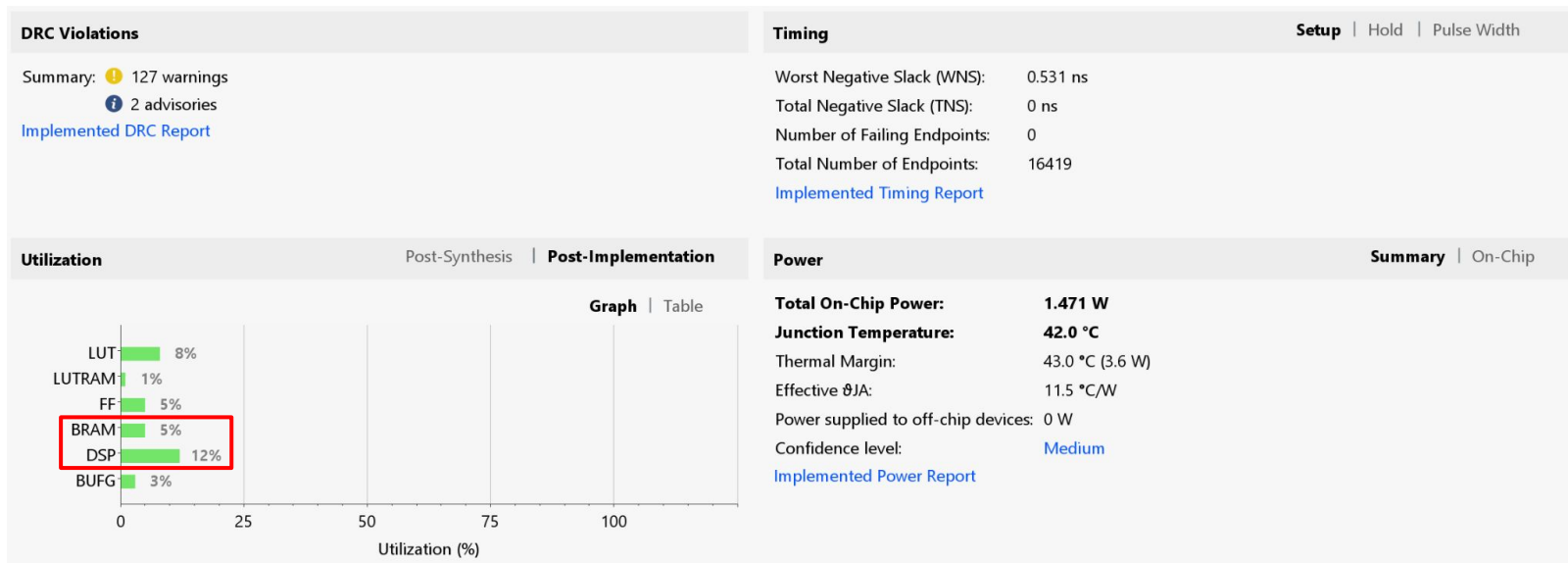
Address map

Name	Interface	Slave Segment	Master Base Address	Range	Master High Address
Network 0 (/axi_dma_0/Data_MM2S, /axi_dma_0/Data_S2MM)					
/axi_dma_0					
/axi_dma_0/Data_MM2S (32 address bits : 4G)					
/axi_dma_0/M_AXI_MM2S					
/processing_system7_0/S_AXI_HP0	S_AXI_HP0	HP0_DDR_LOWOCM	0x0	512M	0x1FFF_FFFF
/axi_dma_0/Data_S2MM (32 address bits : 4G)					
/axi_dma_0/M_AXI_S2MM					
/processing_system7_0/S_AXI_HP0	S_AXI_HP0	HP0_DDR_LOWOCM	0x0	512M	0x1FFF_FFFF
Network 1 (/processing_system7_0/Data)					
/processing_system7_0					
/processing_system7_0/Data (32 address bits : 0x40000000 [1G])					
/processing_system7_0/M_AXI_GP0					
/axi_dma_0/S_AXI_LITE	S_AXI_LITE	Reg	0x4040_0000	64K	0x4040_FFFF
/myip_0/s00_axi	s00_axi	reg0	0x4000_0000	4K	0x4000_0FFF

7 Integration

Synthesis & Implementation result

allocated BRAM & DSP successfully



HAL function flow

1. reset & setup ConvCore (Coeff, size)
2. setup DMA (S2MM)
3. enable ConvCore
4. enable DMA (MM2S)
5. (wait for interrupt for DMA s2mm done)

Registers structure

```
typedef struct {  
    volatile uint32_t CONFIG;          // 0x00: [0] En, [4] Reset (Self-clearing)  
    volatile uint32_t SIZE;            // 0x04: [15:0] Width, [31:16] Height  
    volatile uint32_t STATUS;          // 0x08: [0] Done (Read Only)  
    volatile uint32_t COEFF_R[3];      // 0x0C, 0x10, 0x14  
    volatile uint32_t COEFF_G[3];      // 0x18, 0x1C, 0x20  
    volatile uint32_t COEFF_B[3];      // 0x24, 0x28, 0x2C  
} HP_CONV_REGS;  
  
HP_CONV_REGS *my_hw_ip = (HP_CONV_REGS*) (XPAR_MYIP_0_BASEADDR);
```

Linker script mapping

- allocate Data section (global var) at DDR
- DMA access' DDR directly

Section to Memory Region Mapping

A list of linker sections and their associated memory regions.

SECTION	MEMORY REGION	⋮
.text	ps7_ddr_0_memory_0	▼
.note.gnu.build-id	ps7_ddr_0_memory_0	▼
.init	ps7_ddr_0_memory_0	▼
.fini	ps7_ddr_0_memory_0	▼
.rodata	ps7_ddr_0_memory_0	▼
.rodata1	ps7_ddr_0_memory_0	▼
.sdata2	ps7_ddr_0_memory_0	▼

Compiler macro for DMA

memory alignment for DMA

```
uint32_t SRC_ADDR[IMG_HEIGHT][IMG_WIDTH] __attribute__((aligned (32)));  
uint32_t DST_ADDR[IMG_HEIGHT][IMG_WIDTH] __attribute__((aligned (32)));
```

Cache coherence problem

- DMA & ACC accessed DDR memory without CPU
- Need to flush CPU cache
- S2MM done interrupt => handler makes clear cache symbol

Xilinx HAL - cache clear

```
Xil_DCacheInvalidateRange (SRC_ADDR, IMG_HEIGHT * IMG_WIDTH * PIXEL_SIZE) ;  
Xil_DCacheInvalidateRange (DST_ADDR, IMG_HEIGHT * IMG_WIDTH * PIXEL_SIZE) ;
```

Verification code design

1. Write dummy data on source array
2. Use HW convolution IP, write on destination array
3. Compare with SW code result

Identity filter test

```
Hello world!
Starting convolution IP test!
kernel)
 0, 0, 0
0, 1, 0
0, 0, 0
SRC: 1140e0, DST: 1148e0
DMA Hardware is healthy.
MM2S Start: 0, S2MM Start: 0
Setting Done! Waiting for DMA...
DMA Transfer Complete!
[00][00], HW_DST_ADDR : 01010100, SW_DST_ADDR : 01010100
[00][01], HW_DST_ADDR : 02010200, SW_DST_ADDR : 02010200
[00][02], HW_DST_ADDR : 03010300, SW_DST_ADDR : 03010300
[00][03], HW_DST_ADDR : 04010400, SW_DST_ADDR : 04010400
[00][04], HW_DST_ADDR : 05010500, SW_DST_ADDR : 05010500
[00][05], HW_DST_ADDR : 06010600, SW_DST_ADDR : 06010600
[00][06], HW_DST_ADDR : 07010700, SW_DST_ADDR : 07010700
[00][07], HW_DST_ADDR : 08010800, SW_DST_ADDR : 08010800
[00][08], HW_DST_ADDR : 09010900, SW_DST_ADDR : 09010900
[00][09], HW_DST_ADDR : 0a010a00, SW_DST_ADDR : 0a010a00
[00][10], HW_DST_ADDR : 0b010b00, SW_DST_ADDR : 0b010b00
[00][11], HW_DST_ADDR : 0c010c00, SW_DST_ADDR : 0c010c00
[00][12], HW_DST_ADDR : 0d010d00, SW_DST_ADDR : 0d010d00
[00][13], HW_DST_ADDR : 0e010e00, SW_DST_ADDR : 0e010e00
[00][14], HW_DST_ADDR : 0f010f00, SW_DST_ADDR : 0f010f00
[00][15], HW_DST_ADDR : 10011000, SW_DST_ADDR : 10011000
[01][00], HW_DST_ADDR : 01020200, SW_DST_ADDR : 01020200
[01][01], HW_DST_ADDR : 02020300, SW_DST_ADDR : 02020300
[01][02], HW_DST_ADDR : 03020400, SW_DST_ADDR : 03020400
[01][03], HW_DST_ADDR : 04020500, SW_DST_ADDR : 04020500
[01][04], HW_DST_ADDR : 05020600, SW_DST_ADDR : 05020600
```

sharpen filter test

```

Hello world!
Starting convolution IP test!
kernel)
 0, -1, 0
-1, 5, -1
0, -1, 0
SRC: 1140e0, DST: 1148e0
DMA Hardware is healthy.
MM2S Start: 0, S2MM Start: 0
Setting Done! Waiting for DMA...
DMA Transfer Complete!
[00][00], HW_DST_ADDR : 02020100, SW_DST_ADDR : 02020100
[00][01], HW_DST_ADDR : 04010300, SW_DST_ADDR : 04010300
[00][02], HW_DST_ADDR : 06010500, SW_DST_ADDR : 06010500
[00][03], HW_DST_ADDR : 08010700, SW_DST_ADDR : 08010700
[00][04], HW_DST_ADDR : 0a010900, SW_DST_ADDR : 0a010900
[00][05], HW_DST_ADDR : 0c010b00, SW_DST_ADDR : 0c010b00
[00][06], HW_DST_ADDR : 0e010d00, SW_DST_ADDR : 0e010d00
[00][07], HW_DST_ADDR : 10010f00, SW_DST_ADDR : 10010f00
[00][08], HW_DST_ADDR : 12011100, SW_DST_ADDR : 12011100
[00][09], HW_DST_ADDR : 14011300, SW_DST_ADDR : 14011300
[00][10], HW_DST_ADDR : 16011500, SW_DST_ADDR : 16011500
[00][11], HW_DST_ADDR : 18011700, SW_DST_ADDR : 18011700
[00][12], HW_DST_ADDR : 1a011900, SW_DST_ADDR : 1a011900
[00][13], HW_DST_ADDR : 1c011b00, SW_DST_ADDR : 1c011b00
[00][14], HW_DST_ADDR : 1e011d00, SW_DST_ADDR : 1e011d00
[00][15], HW_DST_ADDR : 31023000, SW_DST_ADDR : 31023000
[01][00], HW_DST_ADDR : 01040300, SW_DST_ADDR : 01040300
[01][01], HW_DST_ADDR : 02020300, SW_DST_ADDR : 02020300
[01][02], HW_DST_ADDR : 03020400, SW_DST_ADDR : 03020400

```

saturation test

overflow & underflow

```

Hello world!
Starting convolution IP test!
kernel)
 0, -1, 0
-1, 65, -1
0, -1, 0
SRC: 1140e0, DST: 1148e0
DMA Hardware is healthy.
MM2S Start: 0, S2MM Start: 0
Setting Done! Waiting for DMA...
DMA Transfer Complete!
[00][00], HW_DST_ADDR : 3e3e3d00, SW_DST_ADDR : 3e3e3d00
[00][01], HW_DST_ADDR : 7c3d7b00, SW_DST_ADDR : 7c3d7b00
[00][02], HW_DST_ADDR : ba3db900, SW_DST_ADDR : ba3db900
[00][03], HW_DST_ADDR : f83df700, SW_DST_ADDR : f83df700
[00][04], HW_DST_ADDR : ff3dff00, SW_DST_ADDR : ff3dff00
[00][05], HW_DST_ADDR : ff3dff00, SW_DST_ADDR : ff3dff00
[00][06], HW_DST_ADDR : ff3dff00, SW_DST_ADDR : ff3dff00
[00][07], HW_DST_ADDR : ff3dff00, SW_DST_ADDR : ff3dff00
[00][08], HW_DST_ADDR : ff3dff00, SW_DST_ADDR : ff3dff00
[00][09], HW_DST_ADDR : ff3dff00, SW_DST_ADDR : ff3dff00

```

```

Hello world!
Starting convolution IP test!
kernel)
 0, -1, 0
-1, 0, -1
0, -1, 0
SRC: 1140e0, DST: 1148e0
DMA Hardware is healthy.
MM2S Start: 0, S2MM Start: 0
Setting Done! Waiting for DMA...
DMA Transfer Complete!
[00][00], HW_DST_ADDR : 00000000, SW_DST_ADDR : 00000000
[00][01], HW_DST_ADDR : 00000000, SW_DST_ADDR : 00000000
[00][02], HW_DST_ADDR : 00000000, SW_DST_ADDR : 00000000
[00][03], HW_DST_ADDR : 00000000, SW_DST_ADDR : 00000000
[00][04], HW_DST_ADDR : 00000000, SW_DST_ADDR : 00000000
[00][05], HW_DST_ADDR : 00000000, SW_DST_ADDR : 00000000
[00][06], HW_DST_ADDR : 00000000, SW_DST_ADDR : 00000000
[00][07], HW_DST_ADDR : 00000000, SW_DST_ADDR : 00000000
[00][08], HW_DST_ADDR : 00000000, SW_DST_ADDR : 00000000
[00][09], HW_DST_ADDR : 00000000, SW_DST_ADDR : 00000000

```

Multiple input test

testing reset logic

```
Hello world!  
Starting convolution IP test!  
kernel)  
  0, -1, 0  
-1, 65, -1  
0, -1, 0  
SRC: 1140e0, DST: 1148e0  
DMA Hardware is healthy.  
MM2S Start: 0, S2MM Start: 0  
Setting Done! Waiting for DMA...  
DMA Transfer Complete!  
Completed without error!  
MM2S Start: 0, S2MM Start: 0  
Setting Done! Waiting for DMA...  
DMA Transfer Complete!  
Completed without error!  
MM2S Start: 0, S2MM Start: 0  
Setting Done! Waiting for DMA...  
DMA Transfer Complete!  
Completed without error!  
MM2S Start: 0, S2MM Start: 0  
Setting Done! Waiting for DMA...  
DMA Transfer Complete!  
Completed without error!
```

Size variance test

maximum width : buffer size(2048)

maximum size : DMA transfer size

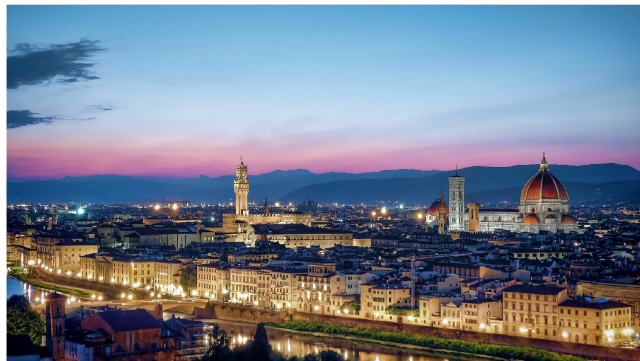
```
Hello world!
Starting convolution IP test!
kernel)
  0, -1, 0
-1, 5, -1
0, -1, 0
Width: 2049, Height: 2333
SRC: 1140e0, DST: 258c9e0
DMA Hardware is healthy.
MM2S Start: 0, S2MM Start: 0
Setting Done! Waiting for DMA...
DMA Transfer Complete!
Error at [00][00], HW_DST_ADDR : 022a0100,
Error at [00][01], HW_DST_ADDR : 04000300,
Error at [01][00], HW_DST_ADDR : 04240300
```

```
Starting convolution IP test!
kernel)
  0, -1, 0
-1, 5, -1
0, -1, 0
Width: 1072, Height: 2333
SRC: 1140e0, DST: 1428c60
DMA Hardware is healthy.
MM2S Start: 0, S2MM Start: 0
Setting Done! Waiting for DMA...
DMA Transfer Complete!
Completed without error!
```

```
Hello world!
Starting convolution IP test!
kernel)
  0, -1, 0
-1, 5, -1
0, -1, 0
Width: 2048, Height: 2048
SRC: 1140e0, DST: 21140e0
DMA Hardware is healthy.
MM2S Start: 0, S2MM Start: 0
Setting Done! Waiting for DMA...
DMA Transfer Complete!
Completed without error!
Hello world!
Starting convolution IP test!
kernel)
  0, -1, 0
-1, 5, -1
0, -1, 0
Width: 2048, Height: 4096
SRC: 1140e0, DST: 41140e0
DMA Hardware is healthy.
MM2S Start: 0, S2MM Start: 0
Setting Done! Waiting for DMA...
DMA Transfer Complete!
Completed without error!
```


Real image processing

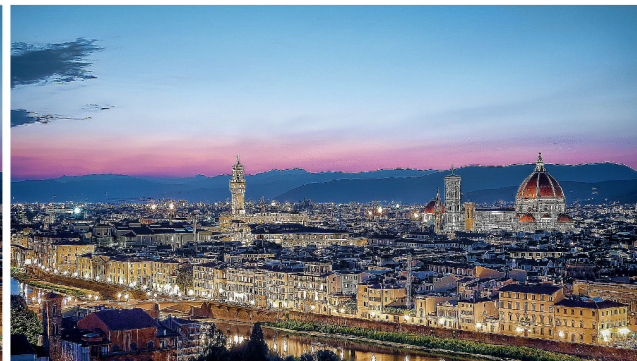
- using JTAG to read/write DDR directly (dow -data, mrd -bin)



original



sharpen



edge enhanced

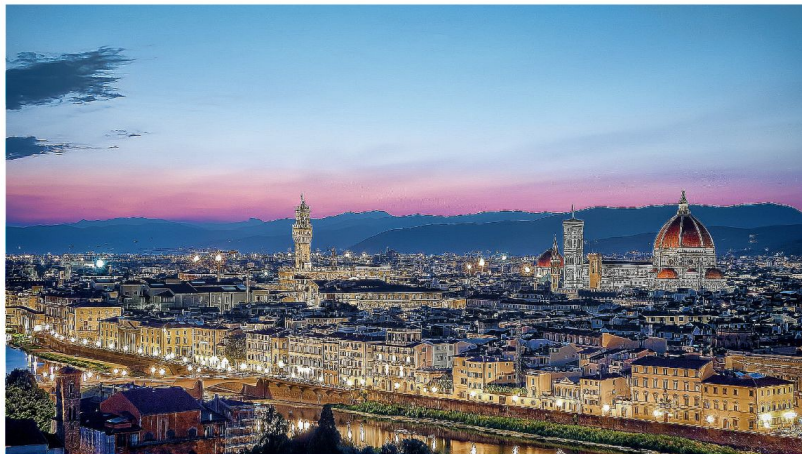
0	-1	0
-1	5	-1
0	-1	0

-1	-1	-1
-1	9	-1
-1	-1	-1

Real image processing

- Test on PYNQ environment

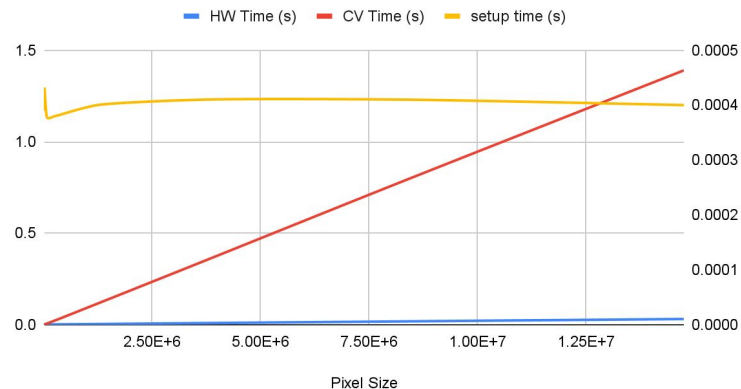
```
# --- [중요] DMA 전송 시퀀스 (Recv -> Send) ---  
  
# 8. DMA 실행  
dma.recvchannel.transfer(out_buffer) # 수신 먼저 예약 (그릇 준비)  
dma.sendchannel.transfer(in_buffer) # 송신 시작  
  
# 9. 완료 대기  
dma.sendchannel.wait()  
dma.recvchannel.wait()  
  
# 10. 결과 확인  
result_img = Image.fromarray(out_buffer)  
display(result_img) # 주피터 노트북에서 즉시 출력
```



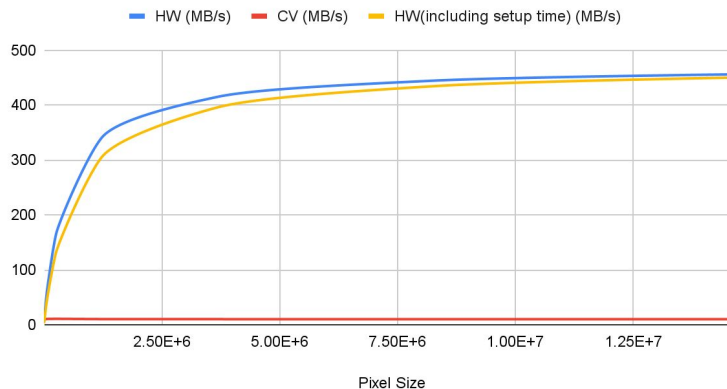
Real image processing

- Test on PYNQ environment
- openCV vs HW convolution
`cv2.filter2D(in_buffer, -1, kernel)`
- benchmark each resolution (convolution IP : 32bit @ 125MHz = 500MBps)

HW vs CV time



Throughput



End.

Filter values

- <https://docs.amd.com/r/en-US/Vitis-Tutorials-AI-Engine-Development/MNIST-ConvNet-Profling-Vector-Load>
- [https://en.wikipedia.org/wiki/Kernel_\(image_processing\)](https://en.wikipedia.org/wiki/Kernel_(image_processing))

DMA HAL

- [Xilinx AXI DMA](#)
-

JTAG read / write

- dow -data
"C:/Users/ehdwn/Desktop/AMBA_vivado/00.Pynq_z2/SW_PYNQ_test/1920x1080_florence.rgb" 0x1140e0
- mrd -bin -file
"C:/Users/ehdwn/Desktop/AMBA_vivado/00.Pynq_z2/SW_PYNQ_test/1920x1080_florence_filtered.rgb" 0x10e60e0 2073600